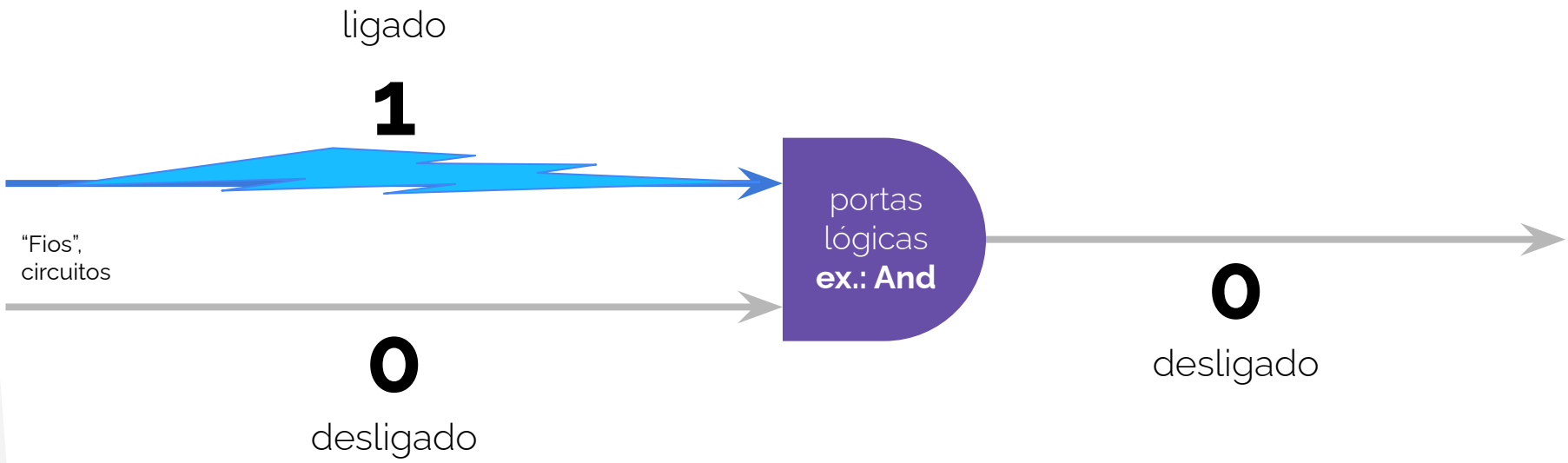
Programação 2

Aula 3 - Revisão:
variáveis, operações básicas, e funções

LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

Recapitulando

Introdução: Problema: Pessoas Programam Máquinas

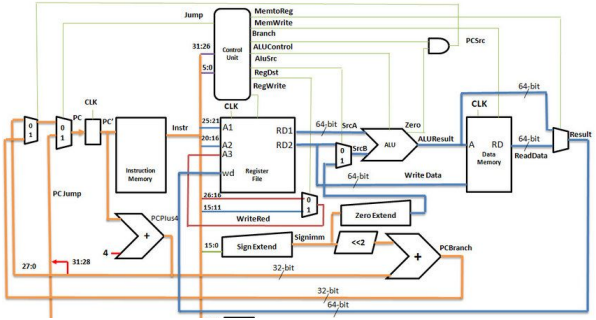


Introdução: Problema: Pessoas Programam Máquinas

Assembly

```
movslq %esi,%rsi
movslq %edi,%rdi
leaq   (%rsi,%rdi,4),%rax
leaq   (%rsi,%rsi,2),%rdx
leaq   (%rdi,%rdx,2),%rdx
movl   0x6011e0(,%rdx,4),%edx
movl   %edx,0x601060(,%rax,4)
retq
```

Processador



Abstrações

Dados: bits (**0**s e **1**s) -> **bytes (hexadecimal)**

Instruções: portas -> **operações (movl)**

LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

Compilação e Máquina Virtual Java (JVM)

Java

```
public class Test {
    Run | Debug
    public static void main(String[] args) {
        System.out.println("test");
    }
}
```

javac

Bytecode

```
????????4?"??Test??java/lang/Object?<init>?
()V?Code
??????LineNumberTable?LocalVariableTable?this?LT
est;?main?
([Ljava/lang/String;)V????java/lang/System???out
?Ljava/io/PrintStream;
??test????java/io/PrintStream???println?
(Ljava/lang/String;)V?args?[Ljava/lang/String;
?SourceFile?Test.java?!
????????????????/??????*??????????
????????????????????
????????7?????? ??????????
??                                        ?!
```

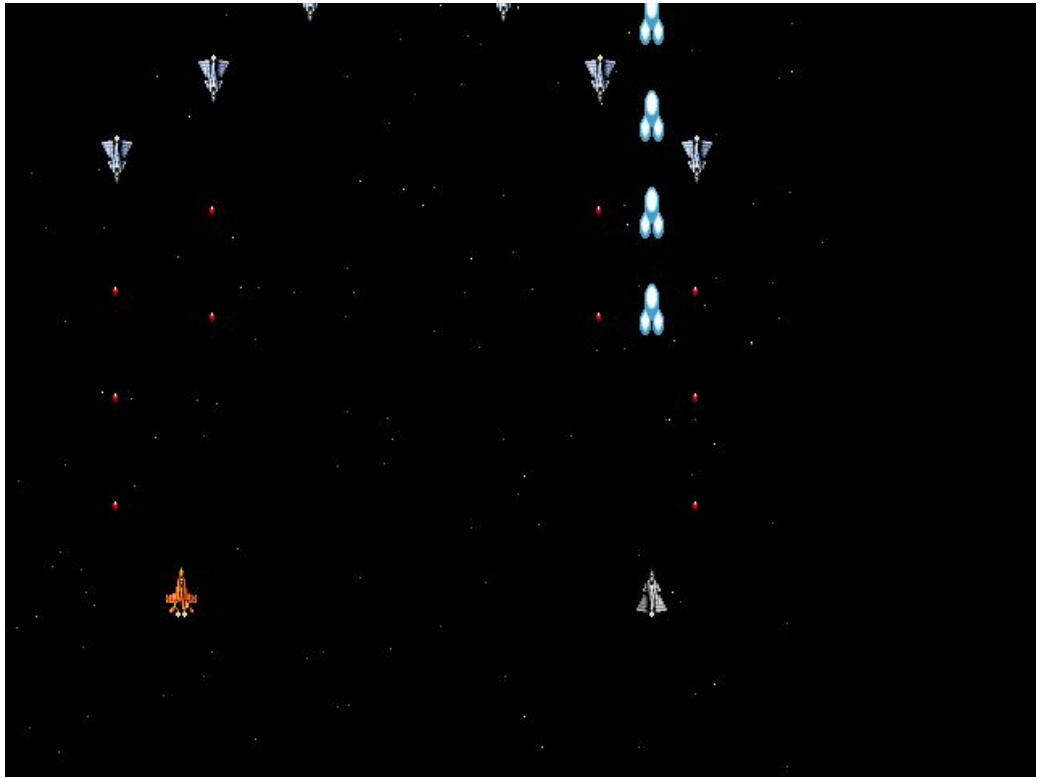
```
7661 2f6c 616e 672f 5374 7269 6e67 3b29
5609 0011 0013 0700 1201 0010 6a61 7661
2f6c 616e 672f 5379 7374 656d 0c00 1400
1501 0003 6f75 7401 0015 4c6a 6176 612f
696f 2f50 7269 6e74 5374 7265 616d 3b08
0017 0100 0474 6573 740a 0019 001b 0700
1a01 0013 6a61 7661 2f69 6f2f 5072 696e
7453 7472 6561 6d0c 001c 001d 0100 0770
7269 6e74 6c6e 0100 1528 4c6a 6176 612f
6c61 6e67 2f53 7472 696e 673b 2956 0100
0461 7267 7301 0013 5b4c 6a61 7661 2f6c
616e 672f 5374 7269 6e67 3b01 000a 536f
7572 6365 4669 6c65 0100 0954 6573 742e
6a61 7661 0021 0001 0003 0000 0000 0002
0001 0005 0006 0001 0007 0000 002f 0001
0001 0000 0005 2ab7 0008 b100 0000 0200
0a00 0000 0600 0100 0000 0100 0b00 0000
0c00 0100 0000 0500 0c00 0d00 0000 0900
0e00 0f00 0100 0700 0000 3700 0200 0100
0000 09b2 0010 1216 b600 18b1 0000 0002
000a 0000 000a 0002 0000 0003 0008 0004
000b 0000 000c 0001 0000 0009 001e 001f
0000 0001 0020 0000 0002 0021
```

JVM

Sistema Operacional

LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

Execução



801 lines (724 sloc) | 22.6 KB

```c
#include <allegro.h>
#include <stdlib.h>

#include "constantes.h"

#define MAX_X         640
#define MAX_Y         480
#define V_MAX_X       0
#define V_MAX_Y       0
#define COLOR_BITS    16

#define DELAY_GIRO    15

        typedef struct {
                int px;
                int py;
                int vx;
                int vy;
                int pintar;
        } Bala;

        typedef struct {
                Bala **balas1;
                BITMAP *bitbal1;
                Bala **balas2;
                BITMAP *bitbal2;
                Bala **balas3;
                BITMAP *bitbal3;
```

[lsfcin/aero-wings: Game coded in 2007 (in between semesters) using Allegro, inspired in AeroFighters and SonicWings! (github.com)]

Introdução: Linha do Tempo

Máquina — **Nível de Abstração** — Pessoa

1950	1960	1970	1980	1990	2000	2010	2020
Assembly Maurice Wilkes	...	**C** Dennis Ritchie	**C++** Bjarne Stroustrup	**Python** Guido van Rossum **Java** James Gosling	**C#** Anders Hejlsberg	**Rust** Graydon Hoare	**Codex** OpenAI

linguagens que vamos explorar na disciplina

Timeline of programming languages - Wikipedia

Make it be the size of the rocketship times 0.75

```
  text.style.left =
rocketship.offsetLeft + 'px';
  text.style.top =
rocketship.offsetTop + 'px';

document.body.appendChild(text
);
  xSpeed = 20;
  setTimeout(function() {
    xSpeed = 5;

document.body.removeChild(text
);
  }, 250);
};


/* Now add an image of an
asteroid:
https://d.newsweek.com/en/full
/1721338/asteroid.jpg?
w=1600&h=1600&q=88&f=9d82d35c9
dw96a82b3fcaf7705eb325b */
var asteroid =
document.createElement('img');
asteroid.src =
'https://d.newsweek.com/en/ful
l/1721338/asteroid.jpg?
w=1600&h=1600&q=88&f=9d82d35c9
dw96a82b3fcaf7705eb325b';
document.body.appendChild(aste
roid);
```

OpenAI GPT-3 e Codex para programação em linguagem natural

LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

Introdução: Linguagens mais Utilizadas no GitHub

Year: 2020 | Quarter: 1

# Ranking	Programming Language	Percentage (YoY Change)	YoY Trend
1	JavaScript	18.703% (-1.406%)	
2	Python	16.238% (-1.654%)	
3	Java	10.938% (+0.538%)	
4	Go	9.005% (+0.978%)	
5	C++	7.423% (+0.040%)	
6	Ruby	6.812% (+0.342%)	
7	TypeScript	6.769% (+1.522%)	
8	PHP	5.127% (-0.458%)	
9	C#	3.835% (+0.141%)	
10	C	3.181% (-0.203%)	

Github Language Stats (madnight.github.io)

Setup

Instalação do VS Code

Instalação do JDK

Instalação dos plugins de Java para o VS Code

Criação do primeiro projeto

Execução

LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

Prática Extra - Chatbot de Cadastro

Crie um chatbot de cadastro de alguma **plataforma/sistema/app** imaginário que você quiser.

O **chatbot deve se apresentar** (usando 'System.out.println') e fazer **perguntas** para o usuário preencher.

Na sequência o chatbot deve **usar as respostas recebidas para interagir** com o usuário e seguir com o preenchimento.

Olá, pode me chamar de Z, sou o chatbot que irá lhe ajudar no cadastro no Zet, o portal de comunicação instantânea entre cidadãos e governo.

Para começarmos, você poderia me dizer como gostaria de ser chamado?
- Lucas

Olá Lucas, é um prazer. Você pode me informa sua idade, por favor?
- 35

Vejo que você é maior de idade, então terá acesso à todas as funcionalidades do sistema...

Conceito de Hoje

Conceito de Hoje

Revisão: variáveis e operações básicas

Tipos e estruturas elementares de dados (ex.: int, float, char)
Operadores (ex.: atribuição, soma)
Instruções condicionais (ex.: if, else)
Instruções de repetição (ex.: while, for)

Constantes

O que são constantes? Dê exemplos

Pi é uma constante

5 também é uma constante

São coisas que não mudam

Constantes

O que são constantes? Dê exemplos

Não muda

Cor Azul

Má fase do Sport

PI

Variáveis

O que são variáveis? Por quê são úteis? Dê exemplos

Entidades que podem possuir valores diferentes em diferentes circunstâncias

Gravidade = 9.8 na terra (nível do mar)

1.6 na superfície da lua

Variáveis

Em computação, o que são variáveis?

Posições na memória, endereçadas por nomes das variáveis

Nome da Variável	Valor da Variável		
Valor da Pessoa 1	R$6.00	Somatório	R$43.00
Valor da Pessoa 2	R$7.00	10%	R$4.30
Valor da Pessoa 3	R$3.00	**Total**	**R$47.30**
Valor da Pessoa 4	R$0.00		
Valor da Pessoa 5	R$27.00		

Sistema binário, tudo são bits

(binário) 000 = 0 (decimal)
(binário) 001 = 1 (decimal)
(binário) 010 = 2 (decimal)
(binário) 011 = 3 (decimal)

(binário) 100 = 4 (decimal)
(binário) 101 = 5 (decimal)
(binário) 110 = 6 (decimal)
(binário) 111 = 7 (decimal)

(binário) 1000 = 8 (decimal)
(binário) 1001 = 9 (decimal)
(binário) 1010 = 10 (decimal)
(binário) 1011 = 11 (decimal)

Tipos e estruturas elementares de dados

Linguagens de programação definem **tipos de dados primitivos/predefinidos**

C

```c
char caractere = 'a';
int numero_inteiro = -100;
float numero_continuo = 13.6f;
double numero_continuo_2 = 13.6;
bool valor_verdade = true;
```

Python

```python
caractere = 'a'
numero_inteiro = -100
numero_continuo = 13.6
valor_verdade = True
sequencia_de_chars = "teste"
```

Java

```java
char caractere = 'a';
int numero_inteiro = -100;
float numero_continuo = 13.6f;
double numero_continuo_2 = 13.6;
boolean valor_verdade = true;
```

C#

```c#
char caractere = 'a';
int numero_inteiro = -100;
float numero_continuo = 13.6f;
double numero_continuo_2 = 13.6;
bool valor_verdade = true;
string sequencia_de_chars = "teste";
```

Diferenças/ Particularidades:

- Python é mais intuitivo, não tem ponto e vírgula
- Python tem menos código, não tem a declaração do tipo da variável
- Python e C# têm sequências de caracteres

Tipos e estruturas elementares de dados

Linguagens de programação definem **tipos de dados primitivos/predefinidos**

C

```c
char caractere = 'a';
int numero_inteiro = -100;
float numero_continuo = 13.6f;
double numero_continuo_2 = 13.6;
bool valor_verdade = true;
```

Python

```python
caractere = 'a'
numero_inteiro = -100
numero_continuo = 13.6
valor_verdade = True
sequencia_de_chars = "teste"
```

Java

```java
char caractere = 'a';
int numero_inteiro = -100;
float numero_continuo = 13.6f;
double numero_continuo_2 = 13.6;
boolean valor_verdade = true;
```

C#

```c#
char caractere = 'a';
int numero_inteiro = -100;
float numero_continuo = 13.6f;
double numero_continuo_2 = 13.6;
bool valor_verdade = true;
string sequencia_de_chars = "teste";
```

Obs.: Python é **fortemente e dinamicamente tipada**, as **demais linguagens são fortemente tipadas**

Esse é um dos motivos de não declararmos o tipo da variável de antemão em Python

Por hora vamos abstrair um pouco isso, já que não vamos trabalhar com Python vamos assumir que as nossas variáveis sempre serão de um único tipo

Tipos e estruturas elementares de dados

Linguagens de programação definem **tipos de dados primitivos/predefinidos**

C

```c
char caractere = 'a';
int numero_inteiro = -100;
float numero_continuo = 13.6f;
double numero_continuo_2 = 13.6;
bool valor_verdade = true;
```

Python

```python
caractere = 'a'
numero_inteiro = -100
numero_continuo = 13.6
valor_verdade = True
sequencia_de_chars = "teste"
```

Java

```java
char caractere = 'a';
int numero_inteiro = -100;
float numero_continuo = 13.6f;
double numero_continuo_2 = 13.6;
boolean valor_verdade = true;
```

C#

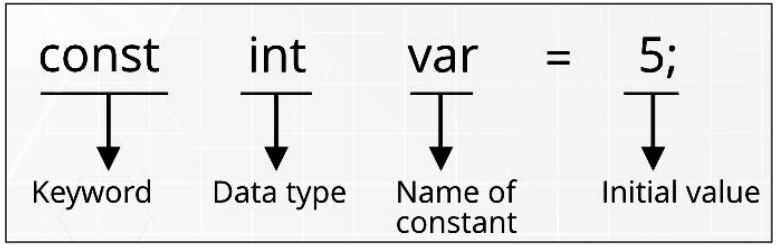
```c#
char caractere = 'a';
int numero_inteiro = -100;
float numero_continuo = 13.6f;
double numero_continuo_2 = 13.6;
bool valor_verdade = true;
string sequencia_de_chars = "teste";
```

Tamanhos dos dados

- chars [8 bits ou 1 byte] = 256 possíveis valores, de 00000000 até 11111111
- bool [8 bits ou 1 byte] = 256 possíveis valores, apenas 2 explorados 00000000 ou 11111111
- int [32 bits ou 4 bytes] = de -2,147,483,648 até 2,147,483,647
- float [32 bits ou 4 bytes] = de 1.175494351 E-38 até 3.402823466 E+38
- double [64 bits ou 8 bytes] = de 2.225073858507201 4 E-308 até 1.797693134862315 8 E+308

Constantes

C, C#



Java

```
private final int INCREMENT;
```

Python

Não existe essa opção

[How do I create a constant in Python? - Stack Overflow]

Constantes e tipos enumeráveis (**enum**)

Com frequência precisamos definir um conjunto de dados ou opções

em que **poderíamos usar números** naturais (1, 2, 3..)
mas que **ficaria mais intuitivo se usássemos nomes** bem definidos

Nestes casos podemos usar o tipo **enum**, como demonstrado abaixo

```
// tipo enum com constantes que representam o estado do jogo
private enum Status { CONTINUE, WON, LOST };
```

constantes `enum` são *implicitamente* `final`.

Operadores

Linguagens de programação definem **operadores** para manipularem os dados

Aritméticos: **+** **-** ***** **/** **%** **++** **--**

```
int variavelA = 13;
int variavelB = 4;
int variavelResultado = variavelA + variavelB; // o valor de variavelResultado após essa linha é 17
variavelResultado = variavelA % variavelB; // o valor de variavelResultado após essa linha é 1
variavelResultado++; // o valor de variavelResultado após essa linha é 2
```

Operadores

Linguagens de programação definem **operadores** para manipularem os dados

Aritméticos: **+** **-** ***** **/** **%** **++** **--**

Comparação: **==** **!=** **>** **<** **>=** **<=**

```
int variavelA = 13;
int variavelB = 4;
bool variavelC = variavelA >= variavelB; // o valor de variavelC após essa linha é true
variavelC = variavelA != variavelB; // o valor de variavelC após essa linha é true
variavelC = variavelC == variavelC; // o valor de variavelC após essa linha é true
```

Operadores

Linguagens de programação definem **operadores** para manipularem os dados

Aritméticos:	**+**	**-**	*****	**/**	**%**	**++**	**--**
Comparação:	**==**	**!=**	**>**	**<**	**>=**	**<=**	
Lógicos:	**&&**	**||**	**!**				
Lógicos [Python]:	and	or	not				

```
bool variavelA = 13 > 5; // o valor de variavelA após essa linha é true
bool variavelB = !variavelA; // o valor de variavelB após essa linha é false
bool variavelC = variavelA && variavelB; // o valor de variavelC após essa linha é false
variavelC = variavelA || variavelB; // o valor de variavelC após essa linha é true
variavelC = !variavelA && variavelB; // o valor de variavelC após essa linha é false
```

Operadores

Linguagens de programação definem **operadores** para manipularem os dados

Aritméticos:	+	-	*	/	%	++	--
Comparação:	==	!=	>	<	>=	<=	
Lógicos:	&&	||	!				
Lógicos [Python]:	and	or	not				
Atribuição:	=	+=	-=	*=	/=	%=	

```
int variavelA = 13;
variavelA += variavelA; // o valor de variavelA após essa linha é 26
variavelA *= variavelA; // o valor de variavelA após essa linha é 676
variavelA %= variavelA; // o valor de variavelA após essa linha é 0
```

Operadores

Linguagens de programação definem **operadores** para manipularem os dados

Aritméticos:	+	-	*	/	%	++	--
Comparação:	==	!=	>	<	>=	<=	
Lógicos:	&&	||	!				
Lógicos [Python]:	and	or	not				
Atribuição:	=	+=	-=	*=	/=	%=	
Bitwise:	&	|	^	~	<<	>>	

```
byte variavelA = 00001111; // o valor da variavelA no sistema decimal é 15
byte variavelB = 01111110; // o valor da variavelB no sistema decimal é 126
byte variavelC = variavelA & variavelB; // em bits, variavelC após essa linha é 00001110, 14
byte variavelD = variavelA ^ variavelB; // o valor de variavelD após essa linha é 01110001, 113
byte variavelE = variavelD << 1; // o valor de variavelE após essa linha é 11100010, 226
```

Instruções condicionais

Programadores podem controlar o **fluxo de execução** a partir de **condições**

if

else if

(**elif** Python)

else

Instruções condicionais

Programadores podem controlar o **fluxo de execução** a partir de **condições**

C, Java, C#

```
int pontos_de_vida_iniciais = 100;
int dano_do_ataque = 17;
int pontos_de_vida_atuais = pontos_de_vida_iniciais - dano_do_ataque;

if(pontos_de_vida_atuais <= 0)
{
    // o personagem não tem mais pontos de vida
    // notificar mensagem de fim do jogo
}
else if (pontos_de_vida_atuais < 25)
{
    // o personagem tem menos que 25 pontos de vida
    // notificar o jogador para ter cuidado
}
else
{
    // notificar o jogador a quantidade de pontos perdidos
}
```

Python

```
pontos_de_vida_iniciais = 100
dano_do_ataque = 17
pontos_de_vida_atuais = pontos_de_vida_iniciais - dano_do_ataque

if pontos_de_vida_atuais <= 0:
    # o personagem não tem mais pontos de vida
    # notificar mensagem de fim do jogo

elif pontos_de_vida_atuais < 25:
    # o personagem tem menos que 25 pontos de vida
    # notificar o jogador para ter cuidado

else
    # notificar o jogador a quantidade de pontos perdidos
```

Instruções condicionais

Programadores podem controlar o **fluxo de execução** a partir de **condições**

C, Java, C#

```
int pontos_de_vida_iniciais = 100;
int dano_do_ataque = 17;
int pontos_de_vida_atuais = pontos_de_vida_iniciais - dano_do_ataque;

if(pontos_de_vida_atuais <= 0)
{
    // o personagem não tem mais pontos de vida
    // notificar mensagem de fim do jogo
}
else if (pontos_de_vida_atuais < 25)
{
    // o personagem tem menos que 25 pontos de vida
    // notificar o jogador para ter cuidado
}
else
{
    // notificar o jogador a quantidade de pontos perdidos
}
```

Descrevendo o código ao lado:

Instruções de repetição

Programadores podem controlar o **fluxo de execução** a partir de **repetições**

while

for

Instruções de repetição

Programadores podem controlar o **fluxo de execução** a partir de **repetições**

C, Java, C#

```
// executando a potencia com a repetição usando while
int i = 1;
int numeroA = 2;
int potencia = 5;

while (i < potencia)
{
  numeroA = numeroA * numeroA;
  i++;
}
```

```
int numeroA = 2;
int potencia = 5;
// executando a potencia com a repetição usando for
for(int i = 1; i < potencia; i++)
{
    numeroA = numeroA * numeroA;
}
```

Python

```
i = 1
numeroA = 2
potencia = 5

while i < potencia:
  numeroA = numeroA * numeroA;
  i += 1
```

Problemática: Chatbot 2.0

E se quiséssemos que o nosso chatbot **permitisse escolhas** de um menu após cada operação?

Como ficaria o nosso código?

Toda vez que o usuário voltasse para o menu, o que aconteceria?

Reuso

Por que programamos?

Reuso

Por que programamos?

*Automatizar processos, diminuir esforço de **tarefas repetitivas***

Então por que deveríamos, nós programadores,
realizarmos atividades repetitivas?

Se existir um problema em um trecho de código, que foi copiado de outro trecho, provavelmente o original também terá um problema

CTRL+C, CTRL+V geralmente é fumaça...

Funções e Métodos

Declaração e Implementação

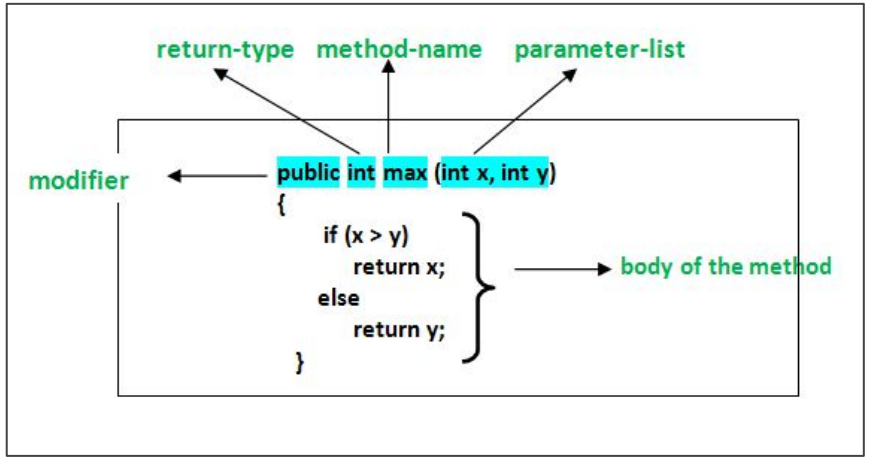
C

```c
char abc(char ch1, char ch2)
{
    char ch3;
    …
    …
    return ch3;
}
```

Python

```python
def sum(a, b):
    return a + b
```

Java, C#



Chamadas

C

```c
char resultado = abc('x', 'y');
```

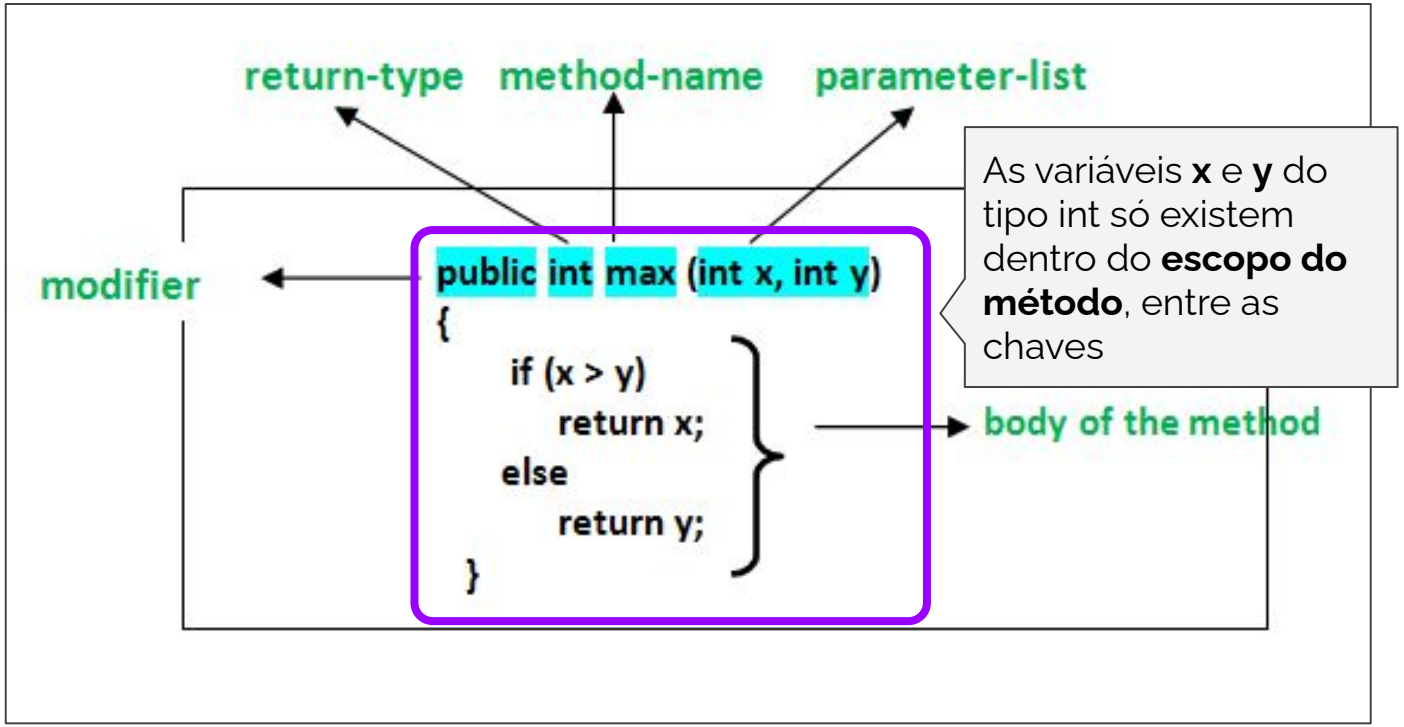
Python

```python
soma = sum(129, 13)
```

Java, C#

```java
int maximo = max(7, 15);
```

Variáveis e Escopo



As variáveis **x** e **y** do tipo int só existem dentro do **escopo do método**, entre as chaves

Métodos: Declaração

Estruturas de declaração dos métodos

```
nivelDeAcesso tipoDeRetorno nomeDoMetodo(tipo parametro1, ...){
    // passos da execução do método
    return variavelDeRetorno; // não é usado para métodos void
}
```

Em que:

```
nivelDeAcesso = public ou private
tipoDeRetorno = tipo de variável (ex.: int, String) ou void (sem retorno)
return = instrução indicando o retorno de uma variável
parametrosN = variáveis locais acessíveis no escopo do método
```

Métodos: Chamada

Estruturas de chamadas de métodos

```
nomeDoObjeto.nomeDoMetodo(parametro1, parametro2, ...)
```

Sobrecarga de Métodos (**overload**)

O mesmo método pode ter implementações diferentes (sem alterar o nome e o tipo de retorno)

```
//declaração e implementação com 2 parâmetros
int max(int a, int b){
    int resultado = a;
    if(a < b){
        resultado = b;
    }
    return resultado;
}

//declaração e implementação com 3 parâmetros
int max(int a, int b, int c){
    return max(a, max(b, c));
}
```

```
Math.max(3, 7);

Math.max(3, 7, 9);
```

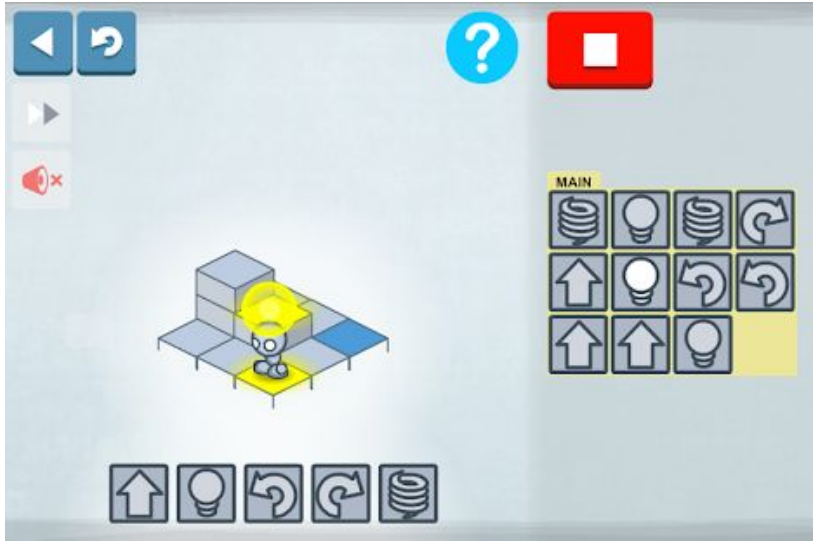
Método *main*

Permite que o programa seja executado sem a criação de nenhum objeto

A máquina virtual responsável por executar o programa Java que se chama Java Virtual Machine (**JVM**) **inicia a execução** pelo método *main*

Prática

Prática Extra: “Zerar” LightBot e SpriteBox coding hour



Lightbot

SpriteBox

Prática Extra: Cálculo de Potência

Crie um código que **calcula a potência de um número**, N^p ex.: 5^3

O cálculo deve ser realizado por um **método que você criou**, e que recebe o valor **N** (ex.: 5) e **p** (ex.: 3) como parâmetros

Você não pode usar as operações de multiplicação nem potência nativas de Java

Desafio: trate os casos especiais em que
p = 1,
p = 0 e em que
p tem valores negativos

LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

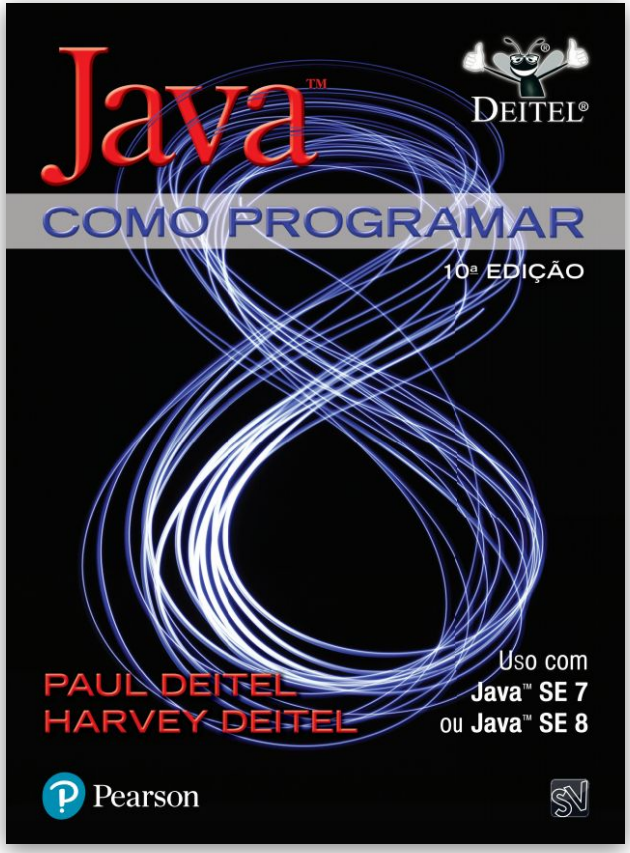
Referências

Java - Como Programar (10a Edição)

1.4 Linguagens de máquina, assembly e de alto nível
1.7 Linguagens de programação
2.7 Aritmética
2.8 Tomada de decisão: operadores de igualdade e operadores relacionais
4.5 A instrução de seleção única if
4.6 Instrução de seleção dupla if...else
4.8 Instrução de repetição while
5.3 Instrução de repetição for

Ferramenta:

Eclipse: eclipse.org/downloads



Glossário

Variável: entidade que nomeia uma porção da memória e é capaz de guardar dados dinamicamente

Identação: tabulação do código, visando a organização do mesmo e no caso específico de python sendo essencial para a **compilação** do código

Compilador: módulo capaz de converter linguagens de alto nível (ex.: C, Java ou Python) para a linguagem de máquina